

Compendio de Desarrollo Backend

Go, PostgreSQL y Docker

Inventory Manager Project

1 de febrero de 2026

Índice

1. Arquitectura y Estructura del Proyecto	2
2. Infraestructura: Docker y Docker Compose	2
3. El Lenguaje: Go (Golang)	2
3.1. Estructuras (Structs) y Tags	2
3.2. Manejo de Errores	3
3.3. Identificador en Blanco (-)	3
4. Base de Datos: SQL y PostgreSQL	3
4.1. Tipos de Consultas	3
4.2. Seguridad y Placeholders	3
4.3. Integridad Referencial	3
5. Integración: HTTP y API REST	3
5.1. Decodificar (Leer JSON)	3
5.2. Codificar (Responder JSON)	4
6. Lógica de Negocio (Ejemplo Venta)	4
7. Cheat Sheet de Comandos	4
8. Arquitectura de Microservicios	5
8.1. Comunicación entre Contenedores	5
8.2. Cliente HTTP en Go (Hacer Peticiones)	5
8.3. Docker Compose Multi-Servicio	5
8.4. Flujo de Venta Distribuido	5

1. Arquitectura y Estructura del Proyecto

Pasamos de un script monolítico a una **Arquitectura Modular**. Esto separa responsabilidades, haciendo el código limpio y mantenible.

- **main.go (El Director):** Solo se encarga de recibir peticiones HTTP (rutas) y decidir qué función llamar. No contiene lógica SQL.
- **database/ (El Almacén):** Paquete separado donde reside la conexión y lógica SQL.
- **Visibilidad (Scope):**
 - **Mayúscula** (ej. InitDB, Producto): Público/Exportado.
 - **Minúscula** (ej. crearTablas): Privado/Interno.

2. Infraestructura: Docker y Docker Compose

Todo corre aislado en contenedores, sin instalar dependencias en el sistema operativo host.

- **Contenedores:** Entornos ligeros para ejecutar la aplicación y la base de datos.
- **docker-compose.yml:** Orquestador que define los servicios:
 1. **app:** Código Go (Puerto 8080).
 2. **db:** PostgreSQL.
- **Red Interna:** Los contenedores se ven por nombre de servicio (host: "db").
- **Volúmenes (Persistencia):**

```
volumes:  
  - postgres_data:/var/lib/postgresql/data
```

Vital para que los datos sobrevivan al reiniciar los contenedores.

3. El Lenguaje: Go (Golang)

3.1. Estructuras (Structs) y Tags

El plano de los datos. Los "Tags" definen cómo se visualizan en JSON.

```
1 type Producto struct {  
2     ID      int      'json:"id" '  
3     Nombre string    'json:"nombre" '  
4     Stock   int      'json:"stock" '  
5 }
```

3.2. Manejo de Errores

Go no usa try/catch. Se verifica el error explícitamente.

```
1 resultado, err := funcionPeligrosa()
2 if err != nil {
3     log.Fatal(err) // Si falla, detenemos la ejecucion
4 }
```

3.3. Identificador en Blanco (-)

Se usa cuando una función devuelve un valor que no necesitamos.

```
1 // Solo importa el error, ignoramos el resultado sql.Result
2 _, err := DB.Exec(query)
```

4. Base de Datos: SQL y PostgreSQL

4.1. Tipos de Consultas

- **DB.Exec:** Para INSERT, UPDATE, DELETE, CREATE.
- **DB.Query:** Para SELECT que devuelve múltiples filas.
- **DB.QueryRow:** Para SELECT de un solo registro (por ID).

4.2. Seguridad y Placeholders

Uso de \$1, \$2 para evitar Inyección SQL.

```
1 query := "SELECT * FROM productos WHERE id = $1"
2 DB.QueryRow(query, idVariable)
```

4.3. Integridad Referencial

Uso de Foreign Keys para relacionar tablas (ej. Ordenes y Productos).

```
1 producto_id INTEGER REFERENCES productos(id)
```

5. Integración: HTTP y API REST

- **GET:** Leer datos. Parámetros en URL (/productos/5).
- **POST:** Crear/Accionar. Datos en Body (JSON).

5.1. Decodificar (Leer JSON)

```
1 var p Producto
2 json.NewDecoder(r.Body).Decode(&p)
```

5.2. Codificar (Responder JSON)

```
1 w.Header().Set("Content-Type", "application/json")
2 json.NewEncoder(w).Encode(p)
```

6. Lógica de Negocio (Ejemplo Venta)

Flujo transaccional implementado:

1. **Recibir:** Orden con ID producto y Cantidad.
2. **Validar:** Existencia y Stock suficiente.
3. **Calcular:** Total = Precio $\times$ Cantidad (Backend autoritativo).
4. **Actualizar:** Restar stock (UPDATE).
5. **Registrar:** Guardar orden (INSERT).
6. **Responder:** Devolver objeto completo.

7. Cheat Sheet de Comandos

docker compose up --build Reconstruye y levanta los contenedores. Usar ante cambios de código.

git checkout -b rama Crea y cambia a una nueva rama.

go mod init Inicia un nuevo proyecto Go.

defer rows.Close() Cierra la conexión a la DB al finalizar la función.

8. Arquitectura de Microservicios

Evolucionamos el sistema monolítico a una arquitectura distribuida, separando la lógica de cobros en un servicio independiente.

8.1. Comunicación entre Contenedores

Utilizamos la red interna de Docker. Los servicios se resuelven por su nombre en `docker-compose.yml`, no por IP ni `localhost`.

- **Inventario:** `http://app:8080`
- **Pagos:** `http://payments:8081`

8.2. Cliente HTTP en Go (Hacer Peticiones)

El backend ya no solo recibe peticiones (Servidor), ahora también las hace (Cliente).

```
1 // 1. Empaquetar datos (Struct -> JSON Bytes)
2 jsonData, _ := json.Marshal(solicitud)
3
4 // 2. Crear buffer (Reader)
5 body := bytes.NewBuffer(jsonData)
6
7 // 3. Realizar el POST a la URL interna de Docker
8 // Notar el host "payments" en lugar de "localhost"
9 resp, err := http.Post("http://payments:8081/cobrar", "application/json",
    , body)
```

8.3. Docker Compose Multi-Servicio

Orquestación de múltiples contenedores simultáneos.

```
1 services:
2   app:
3     ports: ["8080:8080"]
4     depends_on: [db, payments] # Esperar a los vecinos
5
6   payments:
7     build: ./payment-service
8     ports: ["8081:8081"]
```

8.4. Flujo de Venta Distribuido

1. Cliente envía orden a `POST /vender`.
2. Inventario calcula total ($\text{Precio} \times \text{Cantidad}$).
3. **Bloqueante:** Inventario llama a Pagos y espera 2 segundos.
4. Si Pagos responde 200 OK, se guarda en DB.
5. Si Pagos falla, se cancela la transacción (Integridad).